

PILHA

- 1) Um mágico embaralha 5 cartas com os valores A, B, C, D, E e as empilha. Peça aos alunos para simular o processo de colocar as cartas na pilha (push) e depois revelar a ordem em que saem (pop), mostrando que o último a entrar é o primeiro a sair (LIFO).

Desafio extra: inverter a ordem da pilha usando uma segunda pilha.

```
let stack = [];  
let cartas = ['A', 'B', 'C', 'D', 'E'];  
  
// Empilhando  
for (let carta of cartas) {  
    stack.push(carta);  
}  
console.log("Pilha completa:", stack);  
  
// Desempilhando  
while (stack.length > 0) {  
    console.log("Retirando carta:", stack.pop());  
}
```

- 2) Simule as funcionalidades de "Desfazer" e "Refazer" de um editor de texto com duas pilhas: uma para as ações feitas, outra para as ações desfeitas.
Exemplo: Digitar "A", "B", "C", desfazer duas vezes, refazer uma vez.

```
let desfazer = [];  
let refazer = [];  
  
function digitar(letra) {  
    desfazer.push(letra);  
    refazer = []; // limpa o refazer  
}  
  
function desfazerAcao() {  
    if (desfazer.length > 0) {  
        let ultima = desfazer.pop();  
        refazer.push(ultima);  
    }  
}  
  
function refazerAcao() {  
    if (refazer.length > 0) {  
        let voltar = refazer.pop();  
        desfazer.push(voltar);  
    }  
}
```

```
digitar("A");
digitar("B");
digitar("C");
desfazerAcao();
desfazerAcao();
refazerAcao();

console.log("Conteúdo atual:", desfazer.join(''));
```

- 3) Módulos espaciais pousam um sobre o outro. O último a pousar é o primeiro a sair da base. Simule essa sequência com uma pilha e peça para exibir a ordem de retorno à nave.

```
let modulos = [];

modulos.push("Módulo Alpha");
modulos.push("Módulo Beta");
modulos.push("Módulo Gama");

console.log("Ordem de retorno:");

while (modulos.length > 0) {
  console.log("Retornando:", modulos.pop());
}
```

FILA

- 4) Crianças entram na fila da montanha-russa. Cada criança leva x segundos para brincar. Simule a entrada (enqueue) e atendimento (dequeue), imprimindo quem está sendo atendido e o tempo restante.

```
let fila = new Fila();

fila.enqueue("Lucas");
fila.enqueue("Ana");
fila.enqueue("João");

while (!fila.isEmpty()) {
  let tempo = Math.floor(Math.random() * 3000) + 1000;
  console.log(`Brincando: ${fila.dequeue()} por ${tempo}ms`);
}
```

- 5) Clientes fazem pedidos num drive-thru. Crie uma fila que registra os pedidos por nome e imprima a sequência de atendimento.
Extra: calcule tempo total de espera se cada atendimento dura 2 minutos.

```
let pedidos = new Fila();
pedidos.enqueue("Pedido do João");
pedidos.enqueue("Pedido da Maria");
```

```
pedidos.enqueue("Pedido da Carla");

let tempoPorPedido = 2; // em minutos
let total = 0;

while (!pedidos.isEmpty()) {
    let pedido = pedidos.dequeue();
    total += tempoPorPedido;
    console.log(`${pedido} atendido. Tempo acumulado: ${total} minutos`);
}
```

- 6) Implemente uma variação da fila onde clientes com mais de 60 anos passam na frente. (Dica: fazer com duas filas e intercalar atendimento.)

```
let filaNormal = new Fila();
let filaPreferencial = new Fila();

function enfileirar(nome, idade) {
    if (idade >= 60) filaPreferencial.enqueue(nome);
    else filaNormal.enqueue(nome);
}

function atender() {
    if (!filaPreferencial.isEmpty()) return
    filaPreferencial.dequeue();
    else return filaNormal.dequeue();
}

enfileirar("Dona Maria", 65);
enfileirar("Carlos", 30);
enfileirar("Seu João", 70);
enfileirar("Ana", 25);

while (!filaNormal.isEmpty() || !filaPreferencial.isEmpty()) {
    console.log("Atendendo:", atender());
}
```

LinkedList

- 7) Cada nó representa um corredor. Simule a passagem do bastão com insertAtEnd e remova um corredor machucado com removeByValue. Mostre a lista após cada modificação.

```
let corrida = new LinkedList();

corrida.insertAtEnd("Corredor 1");
corrida.insertAtEnd("Corredor 2");
corrida.insertAtEnd("Corredor 3");
```

```
console.log("Passagem do bastão:");
corrida.printList();

console.log("Corredor 2 machucou!");
corrida.removeByValue("Corredor 2");
corrida.printList();
```

- 8) Usuário monta uma lista de compras com `insertAtBeginning` e `insertAtEnd`. Permita remover itens com `removeByValue` e buscar com `find`. Imprima a lista em cada etapa.

```
let compras = new LinkedList();

compras.insertAtBeginning("Arroz");
compras.insertAtEnd("Feijão");
compras.insertAtEnd("Leite");
compras.printList();

compras.removeByValue("Feijão");
compras.printList();

let item = compras.find("Leite");
console.log(item ? "Item encontrado!" : "Não encontrado.");
```

- 9) Cada nó é uma música de uma playlist. Insira músicas no fim, remova uma específica e permita busca por nome. Desafio extra: Adicionar opção de "tocar próxima" (avançar no ponteiro).

```
let playlist = new LinkedList();

playlist.insertAtEnd("Música A");
playlist.insertAtEnd("Música B");
playlist.insertAtEnd("Música C");
playlist.printList();

playlist.removeByValue("Música B");
playlist.printList();

let musica = playlist.find("Música C");
console.log(musica ? "Tocando: " + musica.value : "Música não encontrada");
```

DoublyLinkedList

- 10) Cada vagão é um nó. Simule adicionar vagões à frente e atrás. Mostre a composição do trem indo e voltando com `traverse` e `traverseReverse`.

```
let trem = new DoublyLinkedList();

trem.append("Vagão 1");
trem.append("Vagão 2");
trem.prepend("Locomotiva");
```

```
console.log("Trem normal:");
trem.traverse();

console.log("Trem reverso:");
trem.traverseReverse();
```

- 11) Cada letra digitada é um nó. O cursor pode ir para frente e para trás (simulando o next e prev). Permita inserções e remoções em qualquer posição (insertAt, removeAt).

```
let editor = new DoublyLinkedList();
editor.insertAt("H", 0);
editor.insertAt("e", 1);
editor.insertAt("l", 2);
editor.insertAt("l", 3);
editor.insertAt("o", 4);
editor.traverse(); // Hello

editor.removeAt(1); // Remove o 'e'
editor.traverse(); // Hllo
```

- 12) Use a estrutura para simular o histórico de páginas acessadas. Com traverseReverse, mostre a navegação para trás (voltar página).
Extra: implementar "ir para página específica" usando find.

```
let historico = new DoublyLinkedList();
historico.append("google.com");
historico.append("youtube.com");
historico.append("github.com");

console.log("Últimos acessos (ordem reversa):");
historico.traverseReverse();

console.log("Buscando 'github.com:");
let idx = historico.find("github.com");
console.log(idx >= 0 ? `Encontrado no índice ${idx}` : "Não encontrado");
```